

Estándares de Codificación - SIAPE APK

Nombre

Hector Wanderley Palencia Puentes

Instructor

Oscar Fernando Carvajal Castaño

Centro de Teleinformática y Producción Industrial

Análisis y Desarrollo de Software

Ficha: 3186640

2026

Este documento define los estándares y convenciones de codificación utilizados en el desarrollo de la aplicación móvil SIAPE. Su objetivo es garantizar consistencia, legibilidad y mantenibilidad del código a lo largo del proyecto.

Lenguaje y tecnologías

Tecnología	Versión	Propósito
TypeScript	6.0.3	Lenguaje principal — tipado estático sobre JavaScript
React Native	0.87.0	Framework de desarrollo móvil
React Navigation	7.3.17 (native) / 7.18.9 (native-stack)	Navegación entre pantallas
Axios	1.19.0	Cliente HTTP para consumo de APIs
@react-native-community/datetimepicker	9.1.0	Selector de fechas nativo

Estructura de carpetas

SiapeApp/

src/

screens/ # Una pantalla por módulo

state/ # Estado global (auth.ts)

utils/ # Constantes y tipos por módulo

android/ # Proyecto nativo Android

App.tsx # Punto de entrada y navegación

index.js # Registro de la app

Convenciones de nombres

Archivos:

- Pantallas: NombreModuloScreen.tsx - ejemplo: PedidosScreen.tsx, FacturaNuevaScreen.tsx
- Utilidades: nombreModulo.ts - ejemplo: pedidos.ts, facturas.ts
- Estado global: nombreEstado.ts - ejemplo: auth.ts

Componentes:

- Nombres en PascalCase - ejemplo: PedidosScreen, HeaderActions
- Tipos en PascalCase - ejemplo: Pedido, FacturalItem

Variables y funciones:

- Nombres en camelCase - ejemplo: fetchPedidos, handleGuardar, authState
- Constantes en UPPER_SNAKE_CASE - ejemplo: PEDIDOS_URL, ESTADO_COLORS, TIPOS_PQRS
- Estados de React en camelCase - ejemplo: loading, errorMsg, pedidoSeleccionado

Convenciones de TypeScript

- Todos los archivos usan extensión .tsx para componentes y .ts para utilidades
- Cada módulo define sus tipos en src/utls/nombreModulo.ts
- Los tipos de navegación se definen en App.tsx en RootStackParamList
- Se usan tipos explícitos en props y estados siempre que sea posible
- El índice de acceso [key: string]: unknown se usa para tipos con campos variables

Patrones de desarrollo

Carga de datos:

- Se usa `useFocusEffect` con `useCallback` para recargar datos cada vez que una pantalla recibe el foco
- Se usan estados `loading`, `error` y `datos` en cada pantalla que consume una API
- Se muestra `ActivityIndicator` mientras carga y mensaje de error si falla

Navegación:

- Se usa `navigation.popTo('NombreScreen')` al guardar exitosamente para limpiar el stack
- Se usa `navigation.goBack()` para volver sin limpiar el stack
- Las pantallas de detalle tienen botones de acción en el header usando `navigation.setOptions`

Autenticación:

- El token JWT se almacena en `authState.token` del módulo `src/state/auth.ts`
- Todas las peticiones incluyen el header `Authorization: Bearer ${authState.token}`
- El id del usuario se extrae del JWT decodificando con `atob(token.split('.')[1])`

Formularios:

- Los selectores de opciones se implementan con botón + Modal + FlatList
- Las fechas se seleccionan con `DateTimePicker` de `@react-native-community/datetimepicker`
- Las validaciones se hacen antes del POST/PUT mostrando el error en un Text de color rojo

Comunicación con la API:

- Las URLs base de cada módulo se definen como constantes en `src/utills/nombreModulo.ts`
- Se usa `axios.isAxiosError(err)` para distinguir errores de red de errores del servidor
- Los errores del servidor se muestran con `err.response?.data?.message` con fallback genérico

Estándares de commits

Los commits siguen el estándar Conventional Commits:

tipo(alcance): descripción

Tipo	Uso
feat	Nueva funcionalidad
fix	Corrección de errores
docs	Cambios en documentación
chore	Configuración o mantenimiento

Control de versiones

Parámetro	Valor
Repositorio	https://github.com/0wander1/SiapeApp.git
Rama principal	main
Estrategia	Commits directos a main